GIT & GITHUB

Essential Commands With Practical Examples

A beginner-friendly reference guide

Learn Git. Understand GitHub. Work with confidence.

How to use this guide

This guide explains commonly used Git and GitHub commands in simple English. Read the explanation, study the example, and practise the command in a test repository before using commands that can remove or overwrite work.

Replace placeholders such as <file>, <branch-name>, <commit-hash>, and <repository-url> with the actual names or values from your project.

1. Repository & Setup Commands

git init

Creates a new Git repository in the current folder.

```
git init
```

git clone

Copies an existing remote repository to your computer.

```
git clone https://github.com/username/repository.git
```

git config

Sets Git preferences, such as the name and email recorded in your commits.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

git status

Shows which files are new, modified, staged, or untracked.

```
git status
```

2. Basic Workflow Commands

git add

Stages a specific file so it can be included in the next commit.

```
git add index.html
```

git add .

Stages changes in the current directory and its subdirectories.

```
git add .
```

Important: Run this from the project root if you intend to stage changes throughout the project. Review with git status.

git commit

Saves the staged changes as a new point in the project's history.

```
git commit
```

git commit -m

Creates a commit with a short message describing the changes.

```
git commit -m "Add login page"
```

git commit -am

Stages and commits changes to already tracked files in one step.

```
git commit -am "Update homepage"
```

Important: This does not include new, untracked files. Use git add first for new files.

git push

Uploads local commits to a remote repository.

```
git push
```

git pull

Downloads remote changes and integrates them into the current branch.

```
git pull
```

git fetch

Downloads remote updates without merging them into your current branch.

```
git fetch
```

3. Branching Commands

git branch

Lists local branches. The current branch is marked with an asterisk.

```
git branch
```

git branch <branch-name>

Creates a new branch without switching to it.

```
git branch feature-login
```

git switch

Switches to an existing branch.

```
git switch main
```

git switch -c <branch-name>

Creates a new branch and switches to it.

```
git switch -c feature-login
```

git checkout

Older, widely supported command used to switch branches or restore files.

```
git checkout main
```

git merge

Combines the changes from another branch into the current branch.

```
git switch main
git merge feature-login
```

git branch -d

Deletes a local branch that Git considers safely merged.

```
git branch -d feature-login
```

4. Undo & Restore Commands

git restore

Discards unstaged changes in a file and restores it to the last committed version.

```
git restore index.html
```

Important: This discards uncommitted edits in that file. Make sure you do not need them.

git restore --staged

Removes a file from the staging area but keeps its working changes.

```
git restore --staged index.html
```

git reset

Moves the current branch pointer to another commit. Its effect depends on the option used.

```
git reset HEAD~1
```

git reset --soft

Moves the branch pointer back while keeping changes staged.

```
git reset --soft HEAD~1
```

git reset --hard

Moves the branch pointer and resets staged and working files, discarding changes.

```
git reset --hard HEAD~1
```

Important: DANGEROUS: This can permanently discard local work. Check your status and make a backup before using it.

git revert

Creates a new commit that reverses the changes introduced by an earlier commit.

```
git revert <commit-hash>
```

Important: Often safer than reset for changes that have already been shared with others.

5. Stash Commands

git stash

Temporarily saves tracked working changes so you can work on something else.

```
git stash
```

git stash list

Lists the stashes saved in the repository.

```
git stash list
```

git stash pop

Applies the latest stash and removes it from the stash list.

```
git stash pop
```

git stash apply

Applies a stash but keeps it in the stash list.

```
git stash apply
```

git stash drop

Deletes a stash entry.

```
git stash drop stash@{0}
```

Important: This removes the selected stash. Confirm you no longer need it.

git stash -u

Stashes tracked changes and untracked files.

```
git stash -u
```

6. Remote Repository & GitHub Commands

git remote -v

Displays the remote repository names and their URLs.

```
git remote -v
```

git remote add origin

Connects the local repository to a remote repository, commonly hosted on GitHub.

```
git remote add origin https://github.com/username/repository.git
```

git remote set-url origin

Changes the URL used by the remote named origin.

```
git remote set-url origin https://github.com/username/new-repository.git
```

git push -u origin main

Pushes the main branch and sets origin/main as its upstream for future pushes and pulls.

```
git push -u origin main
```

git push origin <branch-name>

Pushes a selected local branch to the origin remote.

```
git push origin feature-login
```

git pull origin main

Fetches and integrates updates from the remote main branch into the current branch.

```
git pull origin main
```

7. Useful Inspection & Debugging Commands

git log

Displays the commit history.

```
git log
```

git log --oneline

Displays a compact, one-line summary of each commit.

```
git log --oneline
```

git diff

Shows unstaged changes that have not yet been added to the staging area.

```
git diff
```

git show

Displays details about a commit, including the changes it introduced.

```
git show <commit-hash>
```

git blame

Shows who last changed each line of a file and in which commit.

```
git blame index.html
```

git bisect

Uses a binary search through commit history to help identify the commit that introduced a bug.

```
git bisect start
git bisect bad
git bisect good <known-good-commit>
# Test each suggested version, then mark it:
```

```
git bisect good  # or: git bisect bad
git bisect reset
```

git clean -nd

Previews untracked files and directories that would be removed by git clean.

```
git clean -nd
```

git clean -fd

Deletes untracked files and directories from the working tree.

```
git clean -fd
```

Important: DANGEROUS: Deleted untracked files may be difficult to recover. Always preview with git clean -nd first.

8. A Practical Git & GitHub Workflow

The following example shows a basic workflow for starting a project and uploading it to GitHub.

```
# 1. Move into your project folder
cd my-project

# 2. Initialise Git
git init

# 3. Check the project status
git status

# 4. Stage the project files
git add .

# 5. Create the first commit
git commit -m "Initial commit"

# 6. Connect the GitHub repository
git remote add origin https://github.com/username/my-project.git

# 7. Push the main branch to GitHub
git push -u origin main
```

Replace the repository URL with your own GitHub repository URL. If your local branch is not named main, check its name with git branch and use the correct branch name.

9. Common Beginner Mistakes

- **Forgetting to stage files:** A commit only includes changes that have been staged, except when using options such as -a for tracked files.
- **Confusing fetch and pull:** Fetch downloads remote information; pull also integrates those updates into the current branch.
- **Using reset --hard carelessly:** It can discard local changes. Understand the command and make a backup first.
- **Assuming git commit -am includes new files:** It does not stage untracked files. Use git add for new files.
- **Pushing to the wrong branch:** Check the current branch with git branch before pushing.

Final Note

You do not need to memorise every Git command at once. Start with the everyday workflow: git status, git add, git commit, git pull and git push. Practise branching and recovery commands in a test repository before using them on important projects.

D-LABS • Practical. Polished. Digital Experiences.